```
class CX
{
private:
    int a;
    int b;
public:
    CX() = default;      <-1
    CX(int a_, int b_):        <-2
        a(a_),b(b_)
    {}
    int get_a() const
    {
        return a;
    }
    int get_b() const
    {
        return b;
    }
    int foo() const
    {
        return a+b;
    }
};
```

Note that we've explicitly declared the default constructor ❶ as *defaulted* (see section A.3) in order to preserve it as trivial in the face of the user-defined constructor ❷. This type therefore fits all the qualifications for being a literal type, and you can use it in constant expressions. You can, for example, provide a `constexpr` function that creates new instances:

```
constexpr CX create_cx()
{
    return CX();
}
```

You can also create a simple `constexpr` function that copies its parameter:

```
constexpr CX clone(CX val)
{
    return val;
}
```

But that's about all you can do—a `constexpr` function can only call other `constexpr` functions. What you *can* do, though, is apply `constexpr` to